

Lecture 5: Training models

Fitting Models, Optimization, and Algorithms

Firuz Kamalov

MAI 604: Deep Learning

February 22, 2026

Lecture Goals

The primary goals of today's lecture are to understand:

- 1 The concept of optimizing Loss Functions via Gradient Descent.
- 2 The challenges of non-convex optimization (local minima, saddle points).
- 3 Stochastic Gradient Descent (SGD) and its properties.
- 4 Advanced optimization algorithms: Momentum, Nesterov, and Adam.

Contents

- ➊ Gradient Descent
- ➋ Stochastic Gradient Descent
- ➌ Momentum
- ➍ Adaptive Learning Rates
- ➎ Summary

Contents

- ➊ Gradient Descent
- ➋ Stochastic Gradient Descent
- ➌ Momentum
- ➍ Adaptive Learning Rates
- ➎ Summary

Fitting Models

To fit a model, we need a training set $\{x_i, y_i\}$ of input/output pairs.

We seek parameters ϕ for the model $f[x_i, \phi]$ that map inputs to outputs as closely as possible.

Loss Function

We define a loss function $L[\phi]$ that quantifies the mismatch. The goal is to find parameters $\hat{\phi}$ that minimize this loss:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} L[\phi]$$

Gradient Descent Algorithm

Standard methods for training neural networks are iterative. We initialize parameters heuristically and adjust them to decrease the loss.

Step 1: Compute the derivatives (gradient) of the loss with respect to parameters:

$$\nabla L = \frac{\partial L}{\partial \phi}$$

Step 2: Update the parameters moving *downhill*:

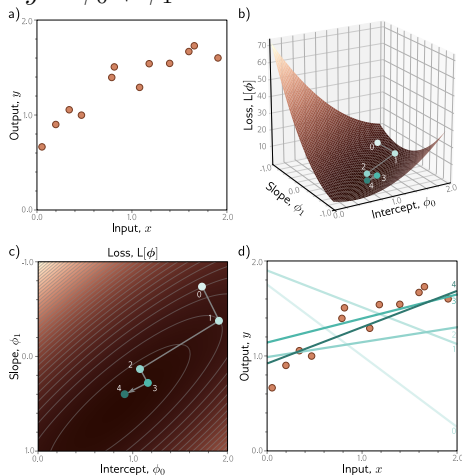
$$\phi \leftarrow \phi - \alpha \cdot \nabla L$$

where α is the positive scalar **learning rate**.

Example: Linear Regression

Consider a 1D linear regression model: $y = \phi_0 + \phi_1 x$.

- Loss function: SSE.
- The surface is *convex* (a smooth bowl).
- It has a single global minimum.
- Gradient descent always converges to the minimum (provided α is not too large).

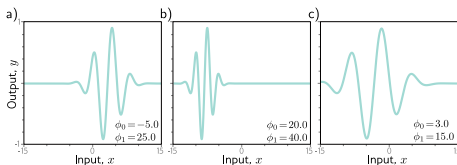


Gradient descent trajectory on a convex surface.

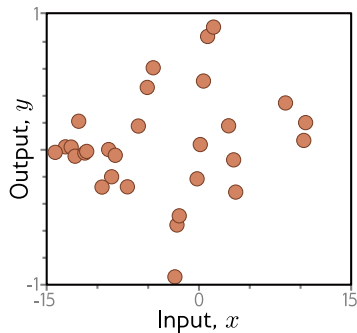
Non-Convex Models: The Gabor Example

Neural network loss functions are usually non-convex. To visualize this, we use a simpler nonlinear "Gabor model":

$$f[x, \phi] = \sin[\phi_0 + 0.06 \cdot \phi_1 x] \cdot \exp\left(-\frac{(\phi_0 + 0.06 \cdot \phi_1 x)^2}{32.0}\right)$$

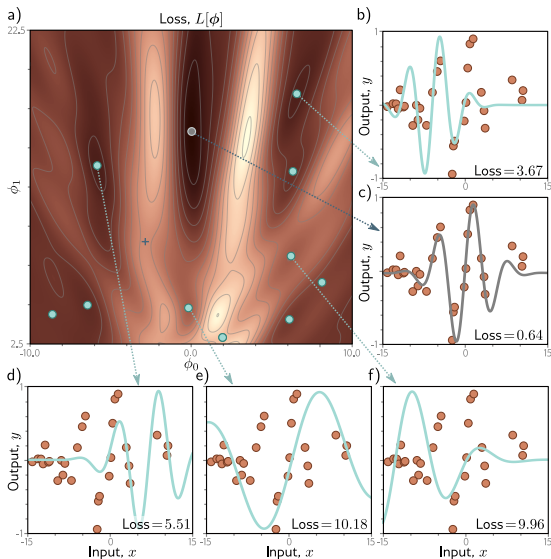


The Gabor function shape.



Training data $\{x_i, y_i\}$.

Local Minima and Saddle Points



Challenges in Non-Convex Optimization:

- **Global Minimum:** The lowest possible loss.
- **Local Minima:** Gradient is zero, but loss is higher than global minimum.
- **Saddle Points:** Gradient is zero; loss increases in some directions and decreases in others.

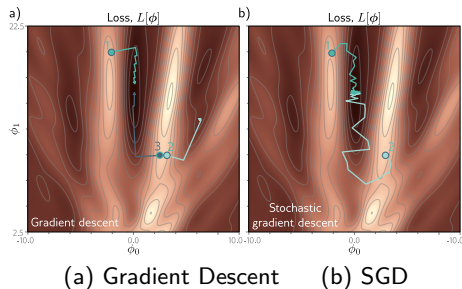
Contents

- ① Gradient Descent
- ② Stochastic Gradient Descent
- ③ Momentum
- ④ Adaptive Learning Rates
- ⑤ Summary

Limitations of Gradient Descent

If we start in the wrong "valley", gradient descent will get stuck in a local minimum.

The final destination is entirely determined by the initialization.



Stochastic Gradient Descent (SGD)

At each iteration t , we choose a random subset of the data (a **minibatch**, $\mathcal{B}_t$) and compute the gradient based on this subset.

$$\phi_{t+1} \leftarrow \phi_t - \alpha \nabla L_t,$$

where $\nabla L_t = \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$.

- **Epoch:** A single pass through the entire training dataset.
- Adds noise to the trajectory.
- Can move temporarily "uphill" on the global loss surface.

Contents

- ① Gradient Descent
- ② Stochastic Gradient Descent
- ③ Momentum
- ④ Adaptive Learning Rates
- ⑤ Summary

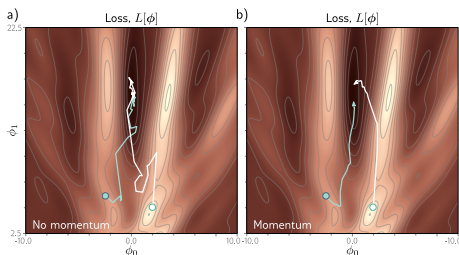
Momentum

SGD can be noisy and zigzag in valleys. **Momentum** smooths the trajectory by maintaining a weighted sum of previous gradients.

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \nabla L_t$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \mathbf{m}_{t+1}$$

- $\beta \in [0, 1)$ controls smoothing.
- Reduces oscillatory behavior.
- Builds speed in directions with consistent gradients.



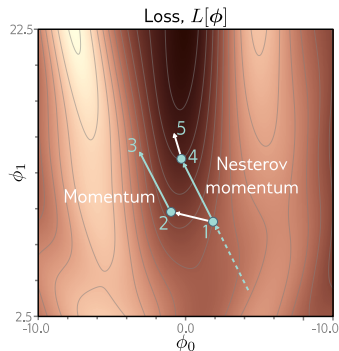
SGD with Momentum.

Nesterov Accelerated Momentum

Nesterov momentum calculates the gradient at the *predicted* future position, rather than the current position.

$$\mathbf{m}_{t+1} \leftarrow \beta \mathbf{m}_t + (1 - \beta) \nabla L_t(\phi_t - \alpha \beta \mathbf{m}_t)$$

The gradient term "corrects" the path provided by momentum alone.



Nesterov update logic.

Contents

- ① Gradient Descent
- ② Stochastic Gradient Descent
- ③ Momentum
- ④ Adaptive Learning Rates
- ⑤ Summary

The Problem with Fixed Learning Rates

If the loss surface changes quickly in one direction and slowly in another (a ravine), a single learning rate α is problematic.

- **Small** α : Slow convergence in flat directions.
- **Large** α : Instability/Oscillation in steep directions.

Solution: Normalize gradients or use adaptive learning rates for each parameter.

Adam (Adaptive Moment Estimation)

Adam maintains momentum for both the gradient ($\mathbf{m}$) and the squared gradient ($\mathbf{v}$).

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \nabla L_t$$

$$\mathbf{v}_{t+1} \leftarrow \gamma \cdot \mathbf{v}_t + (1 - \gamma) (\nabla L_t)^2$$

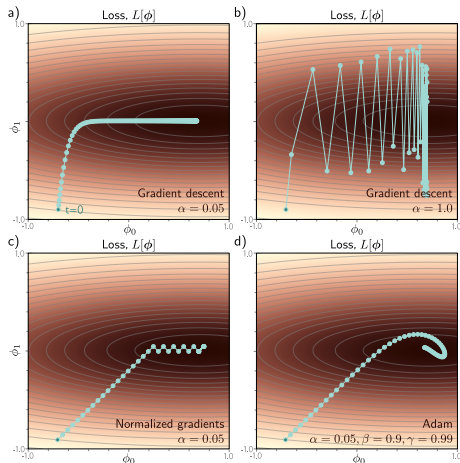
Update rule (simplified):

$$\phi_{t+1} \leftarrow \phi_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \cdot \mathbf{m}_{t+1}.$$

The learning rate becomes *self-adaptive*.

Adam Performance

Adam adapts the step size for each parameter individually. It moves faster in flat directions and slower in steep directions.



Comparison: (a) Slow vertical progress, (b) Instability, (c) Normalized gradients, (d) Adam.

Contents

- ① Gradient Descent
- ② Stochastic Gradient Descent
- ③ Momentum
- ④ Adaptive Learning Rates
- ⑤ Summary

- **Gradient Descent:** Moves downhill on the loss surface. Susceptible to local minima in non-convex problems.
- **SGD:** Uses minibatches. Adds noise, helps escape local minima, and is computationally efficient.
- **Momentum:** Smooths gradients over time to reduce oscillation and accelerate training.
- **Adam:** Combines momentum with adaptive learning rates. Widely used in deep learning.

Typical Hyperparameter Values

Optimizer	Hyperparameter	Typical Value
Standard SGD	Learning Rate (α)	0.01
SGD + Momentum	Learning Rate (α)	0.01
	Momentum (β)	0.9
Nesterov (NAG)	Learning Rate (α)	0.01
	Momentum (β)	0.9
Adam	Learning Rate (α)	0.001
	Momentum (β)	0.9
	Squared grad (γ)	0.999
	Stability (ϵ)	10^{-8}